Modele uczenia maszynowego

Dariusz Put

Plan przedmiotu

1. Wybrane pojęcia związane z uczeniem maszynowym
2. Klasyczne metody uczenia maszynowego
3. Sieci neuronowe (w tym konwolucyjne)
4. Uczenie zespołowe
5. Przetwarzanie tekstu
6. Przetwarzanie szeregów czasowych

Wybrane zasoby Internetu związane z uczeniem maszynowym

Serwis Coursera. Kurs ML autorstwa Andrew Ng
www.coursera.org/learn/machine-learning

Podręcznik użytkownika biblioteki scikit-learn
http://scikit-learn.org/stable/user_guide.html

Interaktywne samouczki dotyczące różnych tematów związanych z przetwarzaniem danych:
https://www.dataquest.io

Zbiory danych można znaleźć w bibliotekach scikit-learn i tensorflow:
https://scikit-learn.org/stable/datasets.html
https://www.tensorflow.org/datasets/catalog/overview?hl=pl
oraz np. na stronie kaggle, na której publikowane są często zadania:
https://www.kaggle.com/
lub innych stronach, m.in.:
https://archive.ics.uci.edu/datasets
https://www.gapminder.org/data/
https://www.kdnuggets.com/datasets/index.html

Plan prezentacji

1. Wybrane pojęcia związane z uczeniem maszynowym
2. Rodzaje zadań uczenia maszynowego
3. Klasyfikacja metod ze względu na sposób uczenia
4. Przygotowanie danych
5. GPU a CPU
6. Biblioteki do uczenia maszynowego

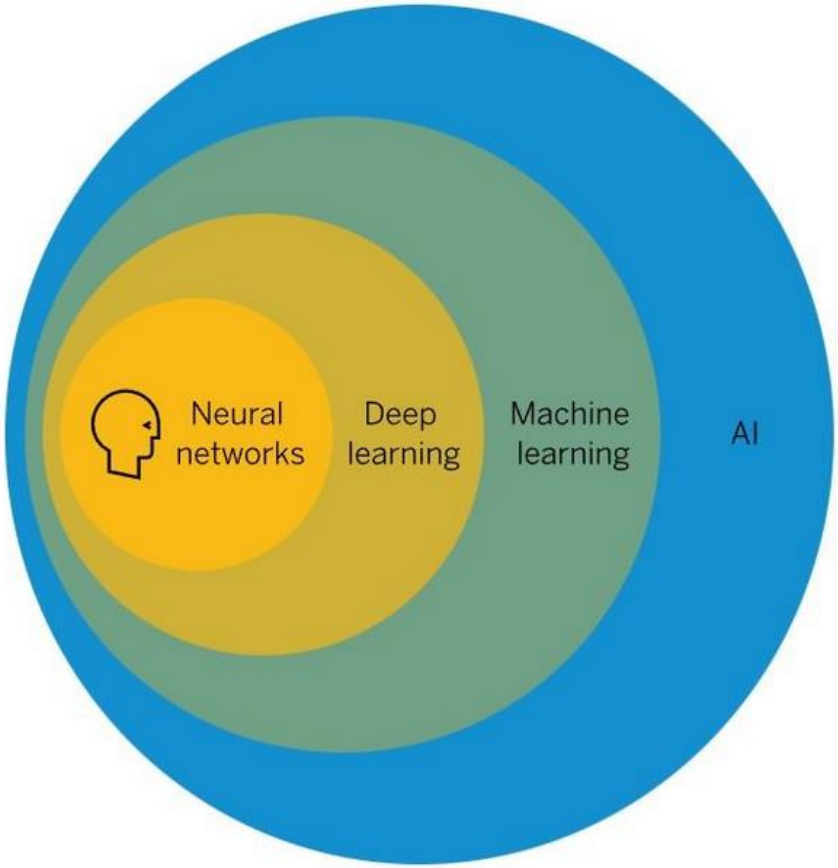
Sztuczna inteligencja (Artificial Intelligence – AI) –
zbiór metod mających na celu automatyzację procesu myślowego standardowo wykonywanego przez ludzi. AI przetwarza dane w celu podejmowania decyzji i formułowania prognoz.

Obecnie AI to szybko rozwijająca się dziedzina, mająca wiele praktycznych zastosowań, m. in.:

- optymalizacja i automatyzacja procesów,
- rozpoznawanie tekstu, obrazów, obiektów, mowy,
- wspomaganie decyzji,
- analiza transakcji i wyłudzeń finansowych,
- segmentacja rynku,
- prognozowanie.

Sztuczna inteligencja jest kategorią nadrzędną wobec wszystkich podzbiorów uczenia maszynowego. Pierwszy podzbiór stanowi uczenie maszynowe, kolejnym w jego obrębie jest głębokie uczenie, a wewnątrz niego znajdują się sieci neuronowe.

Algorytmy **uczenia maszynowego** pozwalają AI uczyć się na podstawie danych i rozwijać inteligencję bez konieczności stosowania dodatkowego oprogramowania.



Pojęcia

Uczenie maszynowe (Machine Learning, ML) to poddziedzina sztucznej inteligencji. Koncentruje się na opracowywaniu algorytmów i modeli umożliwiających komputerom uczenie się i przewidywanie lub podejmowanie decyzji bez konieczności programowania.
Jest to narzędzie, które może zrewolucjonizować różne branże poprzez automatyzację procesów, wyciąganie wniosków z dużych zbiorów danych i usprawnianie procesu podejmowania decyzji.

Machine learning is a subfield of artificial intelligence that focuses on the development of algorithms and models that enable computers to learn and make predictions or decisions without being explicitly programmed.
It is a powerful tool that has the potential to revolutionize various industries by automating processes, extracting insights from large datasets, and improving decision-making.

Machine learning process:

1. Zdefiniowanie problemu

(czego model ma się nauczyć, do czego będzie wykorzystywany?)

2. Zgromadzenie i przygotowanie danych

(dane muszą mieć wysoką jakość, być reprezentatywne, spójne i wolne od szumu)

3. Podział danych na treningowe i testowe

4. Wybór algorytmu trenowania modelu (lub kilku modeli)

(algorytm musi być dostosowany do typu problemu i formy posiadanych danych)

5. Wytrenowanie modelu

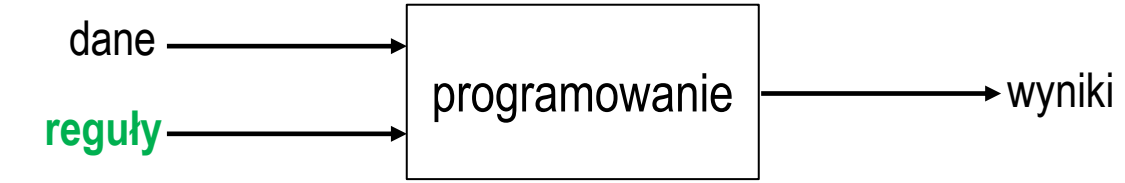
(trenowanie polega na zmniejszaniu błędu i zwiększaniu dokładności predykcji)

6. Oszacowanie poprawności działania modelu i, jeśli to konieczne, dostrojenie i powrót do punktu 5 (lub nawet 4)

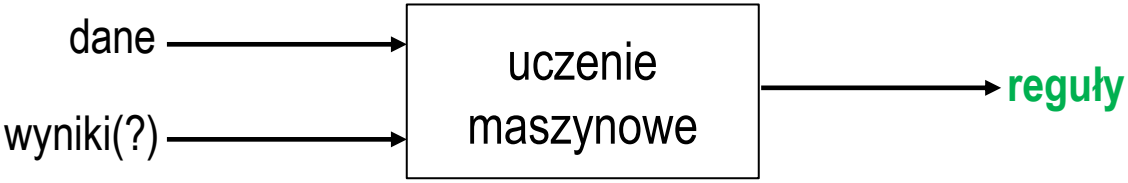
7. Wdrożenie i monitorowanie działania modelu

Uczenie maszynowe:

Człowiek wprowadza dane oraz oczekiwane odpowiedzi a komputer ma utworzyć reguły pozwalające na ich uzyskanie – system uczenia maszynowego jest **trenowany**, służy do odkrywania **reguł**.



podejście deterministyczne



podejście probabilistyczne

Proces uczenia maszynowego (prof. Tom Mitchel):

Program komputerowy uczy się na podstawie doświadczenia (E) w odniesieniu do pewnej klasy zadań (T) i miary efektywności (P), jeśli jego efektywność wykonywania zadania T mierzona za pomocą P poprawia się wraz z doświadczeniem E.

Próbka (ang. *sample*) – element zbioru danych przetwarzany przez model (np. zdjęcie, dane o kredytobiorcy, analizowany tekst).

Klasa (*class*)– zbiór etykiet w problemie klasyfikacji.
Etykieta (*label*) – instancja klasy w problemie klasyfikacji, np. w zadaniu klasyfikacji zdjęć na te, które przedstawiają psy i koty występują dwie etykiety: pies i kot, a klasa to zbiór składający się z dwóch etykiet {pies, kot}.

Wsad (*batch*) – zbiór próbek jednocześnie przetwarzanych przez model. Najczęściej liczba próbek jest potęgą liczby 2. Wsad jest używany do wyznaczenia jednej aktualizacji wag modelu.

Epoka (*epoch*) – pojedyncze przetworzenie całego zbioru danych.

Wagi (**parametry** warstwy, *parameters*) to zbiór liczb. Uczenie maszynowe polega na znajdowaniu wartości wag tak, aby sieć poprawnie mapowała dane wejściowe na wyjściowe. Wagi nazywane są parametrami modelu.
Hiperparametry (*hyperparameters*) – atrybuty modelu dostrajane przez ekspertów (twórców modelu).

Wagi (parametry) są dostrajane automatycznie w procesie uczenia, hiperparametry przez ekspertów lub dobierane przy pomocy znanych metod.

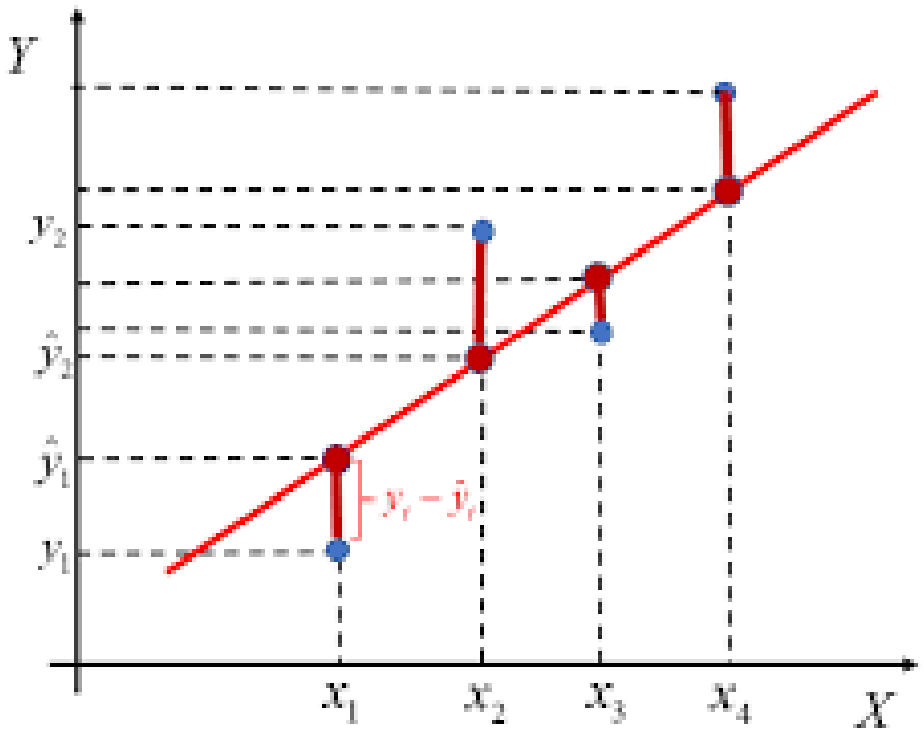
Jednym z wyzwań uczenia maszynowego jest **zautomatyzowanie procesu dostrajania hiperparametrów** modelu. Podejmowano takie próby, jednak w praktyce okazało się, że wiązało się to z dużymi problemami z wydajnością.
Ostatnie prace wykazały, że przy dużej mocy obliczeniowej algorytmy mogą przewyższać w dostrajaniu modeli wydajność uzyskiwaną przez ekspertów.

Dostrajania hiperparametrów modeli to umiejętność charakteryzująca ekspertów w dziedzinie uczenia maszynowego.

Dane wejściowe – dane dostarczane do modelu (x_i).

- **Dane docelowe** – rzeczywiste wartości, które zostałyby przewidziane przez model idealny (y_i).
- **Dane wyjściowe** (przewidywane) – wyniki generowane przez model ($\hat{y}_i$).

Wartość straty (błąd przewidywań) – miara odległości między wartościami wyjściowymi a docelowymi ($y_i - \hat{y}_i$)



Tensor – podstawowa struktura danych w systemach uczenia maszynowego.

Tensor to uogólnienie macierzy:

- tensor zerowymiarowy to skalar, np. 3.56
- tensor jednowymiarowy to wektor, np. [3, 5, 6, 2, 56, 21]
- tensor dwuwymiarowy to macierz
- tensory wielowymiarowe

Atrybuty tensorów:

- liczba osi (ranga) – np. macierz ma dwie osie – jest to tensor o randze 2,
- kształt – krotka wartości całkowitoliczbowych określających liczbę wymiarów poszczególnych osi tensora. Np.
  – skalar (tensor zerowymiarowy) opisuje pusta krotka kształtu ()
  – wektor (tensor o randze 1) ma kształt (m,), np. wektor [3, 5, 6, 2, 56, 21] ma kształt (6,)
  – macierz ma kształt (n, m)
- typ danych, zwykle rzeczywisty (np. float, a w numpy float32, float64),
  może być całkowity (int; int8, int32, int64)
  lub tekstowy (string)

Przykład:

2.3 4.2 3.3
4.1 4.6 0.2

Jest tensorem:

o randze (liczbie osi)?	2
ma kształt?	(2,3)
dane są liczbami typu?	float

Przykłady tensorów danych:

Tensory dwuwymiarowe (próbki, cechy) – najczęściej wykorzystywane.
Przykład:
zbiór danych o 100 tys. osób (próbek), z których każda jest charakteryzowana przez trzy atrybuty (wiek, wzrost, waga) będzie zapisany w postaci tensora o kształcie(?)
(100000, 3).
Liczba osi (ranga) tego tensora jest równa(?)
2.

Tensory trójwymiarowe (próbki, cechy, czas) – do analizy szeregów czasowych.
Przykład:
co minutę zapisywana jest cena minimalna i maksymalna akcji w postaci wektora (czas, cena minimalna, cena maksymalna),
dane z całego dnia (480 minut) zapisywane będą w postaci tensora dwuwymiarowego (480, 3)
a z całego roku, przy założeniu, że w roku jest 250 dni, w których giełda jest otwarta, zapiszemy w postaci tensora o kształcie(?)
(250, 480, 3)

Przykłady tensorów danych, c.d.:

Tensory czterowymiarowe (próbki, wysokość, szerokość, głębia koloru) – do rozpoznawania obrazów, np. dla 500 obrazów kolorowych o rozmiarze 50x60 pikseli konieczny jest tensor o kształcie(?)
(500, 50, 60, 3) – gdzie ostatnia oś (o wymiarze 3) wskazuje nasycenie kolorami (RGB).

Przykłady tensorów danych, c.d.:

Tensory pięciowymiarowe (próbki, klatki, wysokość, szerokość, głębia koloru).
Przykład:
film o rozdzielczości 144x256 trwający 60 sekund, o częstotliwości 4 klatek na sekundę składałby się z 240 klatek (4*60).
Próbkę pięciu filmów można zapisać w tensorze o kształcie(?)
(5, 240, 144, 256, 3)

Wybrane operacje na tensorach:

- **zmiana kształtu tensora (reshape)** – zmiana rozmiaru (liczby osi, rangi) tensora, np. tensor jednowymiarowy może zostać przekształcony na dwuwymiarowy. Tensor przekształcony ma taką samą liczbę elementów jak początkowy.
  Przykład:
  tensor (24,) może zostać zamieniony na (8,3) lub (4,2,3)
- **flattening** – operacja reshape polegająca na przekształceniu tensora wielowymiarowego w jednowymiarowy
- **rzutowanie** (*broadcasting*) – automatyczna zmiana wymiarów tensorów w celu wykonania operacji arytmetycznych na tensorach o różnych kształtach. Na przykład, mając tensory o kształtach (3, 4) i (1, 4) broadcasting automatycznie rozszerzy tensor o kształcie (1, 4) do kształtu (3, 4), poprzez powielenie jego pierwszego wiersza trzykrotnie. Dzięki temu można wykonać różne operacje między tymi dwoma tensorami.
- **split** – podział tensora na kilka mniejszych tensorów wzdłuż określonej osi (np. tensor o wymiarach 8*4 może zostać podzielony na dwa tensory o wymiarach 6*4 i 2*4).

Przykład:

```
import numpy as np
x=np.array([1,2,3,4,5,6])
x.shape
```

wynik (6,) tensor jednowymiarowy

```
y=np.array([[1,2,3,4,5,6]])
y.shape
```

wynik (1,6) tensor dwuwymiarowy

```
z=np.array([[[1,2,3,4,5,6]]])
z.shape
```

wynik (1,1,6) tensor trójwymiarowy

```
z1=z.reshape(6,)
z1
```

wynik [1,2,3,4,5,6]

```
z1.shape
```

wynik (6,)

Do modelu należy dostarczyć tensor o spodziewanych przez model wymiarach

Rodzaje zadań uczenia maszynowego:

Klasyfikacja binarna (ang. *binary classification*)
zdanie klasyfikacji, w którym każda próbka jest przyporządkowywana do jednej z dwóch rozłącznych kategorii.
Przykład:
Rozpoznanie, czy na zdjęciu jest pies czy kot.
Klasyfikacja binarna: klasa jest zbiorem dwuelementowym (etykietami są pies i kot).
Klasyfikacja jednoetykietowa: wynikiem klasyfikacji jest przyporządkowanie do jednej z kategorii, na zdjęciu jest pies **lub** kot.

Rodzaje zadań uczenia maszynowego, c.d.:

Klasyfikacja wieloklasowa (ang. *multiclass classification*)
zadanie klasyfikacji, w którym każda próbka jest przyporządkowywana do jednej kategorii ze zbioru zawierającego więcej niż dwie.
Przykład:
Rozpoznanie, jaka cyfra znajduje się na zdjęciu.

Rodzaje zadań uczenia maszynowego, c.d.:

Klasyfikacja wieloetykietowa (**wielokategorialna, wieloznakowa,** ang. *multi-label classification*) każda próbka może być przyporządkowana do wielu klas (każdej próbce można przypisać wiele etykiet).

Przykład:

na zdjęciu znajduje się kilka zwierząt, więc każde zdjęcie może być przypisane do wielu kategorii.

Rodzaje zadań uczenia maszynowego, c.d.:

Regresja (uczenie nadzorowane) – próbuje znaleźć wartości parametrów funkcji najlepiej opisującej zbiór danych. Celem jest minimalizacji funkcji kosztu (błędu). Typową funkcją kosztu jest błąd średniokwadratowy, czyli kwadrat różnicy między wartością oczekiwaną a prognozowaną (ang. RMSE – root square mean error).

Regresja skalarna:
wartości docelowe mają charakter ciągłych wartości skalarnych.
Przykład: przewidywanie cen towaru.

Regresja wektorowa:
wartości docelowe są wektorem wartości ciągłych.
Przykład: przewidywanie cen koszyka towarów.

Regresja jednoczynnikowa:
badamy zależność zmiennej objaśnianej od jednej zmiennej objaśniającej (niezależnej).

Regresja wieloraka (wieloczynnikowa):
do prognozowania wyniku wykorzystujemy wiele cech (zmiennych niezależnych).
Przykład: do prognozowania ceny towaru wykorzystujemy różne atrybuty towaru i otoczenia (np. ceny innych podobnych towarów, dochody potencjalnych nabywców).

Przykład:
Zadanie regresji wieloczynnikowej: oszacowanie ceny domu na podstawie różnych cech.
Mamy **dwupiętrowy**, **25**-letni dom o powierzchni **100** m.kw., z **trzema** łazienkami. Ponadto dzielimy miasto na 10 obszarów (dzielnic) i nasz dom znajduje się w obszarze oznaczonym liczbą **7**. Możemy więc sparametryzować dom pięciowymiarowym wektorem:
x=[100, 25, 3, 2, 7].
Załóżmy, że przybliżona wartość domu wynosi $200 000. W regresji liniowej chodzi o znalezienie wektora **w** = [w1, w2, w3, w4, w5] takiego, że
100*w1 + 25*w2 + 3*w3 + 2*w4 + 7*w5 = 200000.
W przypadku danych o dużej liczbie domów wraz z ceną algorytm uczenia potrafi dopasować wektor **w** aby różnica między ceną oszacowaną a oczekiwaną była najmniejsza.

Po oszacowaniu funkcji straty (średniego błędu kwadratowego) liczymy jej pochodną. Pochodna wskazuje kierunek, w którym funkcja rośnie lub maleje. Zmiana wartości wektora w przeciwnym kierunku do pochodnej poprawia dokładność szukanej funkcji – zmniejsza stratę sieci.

Musimy jeszcze zdecydować, jak szybko chcemy się poruszać. Zbyt duży krok może spowodować wyjście poza minimum. Wartość tego kroku określa się jako **tempo uczenia się** (*learning rate*). Wielokrotne powtórzenie tego procesu pozwala wygenerować coraz lepszą wartość **w**. Należy podkreślić, że jeśli funkcja nie jest wypukła (czyli nie ma jednego minimum) proces ten działa lokalnie. Jeśli funkcja ma wiele minimów lokalnych algorytm ten może zatrzymać się na jednym z nich.

Klasyfikacja metod uczenia maszynowego ze względu na sposób uczenia:

- **uczenie nadzorowane** (*supervised learning*) – poznawanie zależności między danymi treningowymi a znanymi etykietami danych (np. klasyfikacja binarna, klasyfikacja wieloklasowa, regresja skalarna, detekcja obrazów, predykcja)

Zbiór treningowy (y znane):

$x_{11}, x_{12}, x_{13}, \ldots, x_{1n}$ y_1

$x_{21}, x_{22}, x_{23}, \ldots, x_{2n}$ y_2

…

$x_{m1}, x_{m2}, x_{m3}, \ldots, x_{mn}$ y_m

Model się uczy

Zbiór testowy (y szacowane przez model):

$x_{t11}, x_{t12}, x_{t13}, \ldots, x_{t1n}$ $y_{t1}(?)$

…

Model szacuje wartości y

Klasyfikacja metod uczenia maszynowego ze względu na sposób uczenia, c.d.:

- **uczenie nienadzorowane** (*unsupervised learning*) – określanie transformacji danych wejściowych bez pomocy docelowych etykiet. Najczęściej spotykanym zastosowaniem uczenia nienadzorowanego są redukcja liczby wymiarów i grupowanie (klasteryzacja),
- **uczenie częściowo nadzorowane** (*semi-supervised learning*) – w procesie uczenia wykorzystywane są etykiety wygenerowane przez system na podstawie danych wejściowych (zwykle stosuje się do tego algorytm heurystyczny).
  Przykładowo, system do przechowywania zdjęć:
  Posiadamy zdjęcia rodzinne i podpisujemy parę z nich imionami bliskich. Następnie algorytm sam rozpoznaje oznaczone osoby na pozostałych zdjęciach i przypisuje im odpowiednie imiona.
  System sam proponuje odpowiedzi i jest w stanie stworzyć ogólne wzorce.
  Zastosowanie: rozpoznawanie mowy i obrazu, klasyfikacja stron internetowych,
- **uczenie przez wzmacnianie** (*reinforcement learning*) – agent otrzymuje pewne informacje na temat środowiska, w którym pracuje (dozwolone działania, reguły i stwierdzenia) i uczy się wyboru akcji, które prowadzą do maksymalizacji nagrody. W charakterze sygnału wzmacniającego wykorzystywany jest wynik gry albo przyznanie lub utrata punktów. W uczeniu przez wzmacnianie podstawową koncepcją jest korzyść. Algorytm dąży do maksymalizacji korzyści. System nie jest trenowany za pomocą treningowego zbioru danych lecz uczy się metodą prób i błędów. Technika ta jest wykorzystywana głównie w grach (szachy, Go), nie odniosła znaczących sukcesów poza grami. Ma potencjał do zastosowania w pojazdach autonomicznych, robotyce, zarządzaniu, edukacji.
  Uczenie przez wzmacnianie można porównać do osoby, która korzysta z urządzenia bez czytania instrukcji obsługi – uczy się korzystać na zasadzie prób i błędów.

Przykład uczenia nadzorowanego:
Oprogramowanie emailowe przekierowuje niektóre emaile do spamu. Do algorytmu uczenia nadzorowanego zostaje przekazana duża liczba emaili oznaczonych jako „spam” i „nie spam”. Algorytm, korzystając z tych informacji, dokona prognozy, czy nowa wiadomość jest spamem czy nie. Początkowo jakość klasyfikacji będzie niska. Jeśli jednak użytkownik będzie wskazywał wiadomości jako „to jest spam” albo „to nie jest spam”, oprogramowanie będzie coraz częściej klasyfikować je poprawnie. Oznaczenie wiadomości przez użytkownika jako „spam” lub „nie spam” to etykieta wiadomości.

W uczeniu nadzorowanym kategorii (etykiet) może być wiele (np. rozpoznawanie obrazów).

Przykład uczenia NIEnadzorowanego:
Dostarczamy dane o klientach wraz z informacją o robionych przez nich zakupach i oczekujemy, że model podzieli klientów na rozdzielne grupy przyjmując jakąś miarę podobieństwa.

Podział metod uczenia maszynowego ze względu na sposób przesyłania strumienia danych: uczenie wsadowe i przyrostowe.

Uczenie wsadowe (ang. *batch learning*)

Do nauki wykorzystywane są wszystkie dostępne dane.

System najpierw jest uczony a następnie zostaje wdrożony do cyklu produkcyjnego i już więcej nie jest trenowany. Tego rodzaju uczenie zwykle odbywa się offline.

Jeśli pojawią się nowe dane, model musi być wytrenowany od podstaw przy użyciu wszystkich dostępnych danych a następnie stary model zostaje zastąpiony nowym.

Uczenie przyrostowe (ang. *online learning*)

System jest trenowany na bieżąco poprzez sekwencyjne dostarczanie danych.

The output of a machine learning algorithm is entirely dependent on the data it is exposed to.
Change the data, change the result.

Kimberly Nevala – the director of Business Strategies for SAS Best Practices

Rodzaje danych (jedna z wielu klasyfikacji):

1. Liczbowe (ilościowe, numeryczne, ang. *numerical*)
   - dyskretne (całkowite)
     – przedziałowe – różnica (odległość) między wartościami jest istotna, np. liczba produktów
     – porządkowe (ang. *ordinal*) – kolejność jest istotna, ale różnica między wartościami nie, np. wysoki ma wartość 2, średni 1 a niski 0)
   - ciągłe (rzeczywiste, np. cena produktu)
2. Skategoryzowane (kategoryzujące, jakościowe, kategorialne, ang. *categorical*) – dane nie będące liczbami. Do tego rodzaju danych zaliczamy:
   - nominalne (ang. *nominal*) – nie mogą być porównane między sobą, nie występuje kolejność (np. kolor, zawód, miejscowość, jednostka miary)
   - porządkowe – porównywalne między sobą (np. ocena A jest lepsza, niż ocena B; wzrost może być wysoki, średni lub niski, test może być trudny, neutralny lub łatwy)
   - dychotomiczne (binarne, ang. *dichotomous*) – dane mające dwie kategorie (np. tak/nie; wynik testu pozytywny/negatywny; kobieta/mężczyzna; egzamin zdany/niezdany). Mogą to być też dane liczbowe (0 lub 1).

Do modeli uczenia maszynowego można przekazywać jedynie dane liczbowe. Pozostałe należy przetworzyć do postaci liczbowej.

Przygotowując **dane** należy przestrzegać następujących zasad:

1. **Reprezentatywność**. Zbiór treningowy i testowy muszą być reprezentatywne. Załóżmy, że mamy uporządkowany zbiór obrazów cyfr (0-9). Jeśli podzielimy dane na zbiór treningowy i testowy w taki sposób, że pierwszych 80% obrazów znajdzie się w zbiorze treningowym a pozostałe 20% w zbiorze testowym, to zbiór treningowy będzie zawierał tylko cyfry 0-7 a testowy 8-9.
2. **Czas**. Jeśli prognozujemy przyszłość na podstawie danych historycznych, nie należy dokonywać losowania podczas podziału na zbiory. Zbiór testowy musi zawierać dane późniejsze niż treningowy.
3. **Redundancja**. W realnych danych zdarza się, że jakieś obserwacje pojawiają się wielokrotnie. W wyniku losowania może wtedy dojść do sytuacji, że w zbiorze treningowym, walidacyjnym i testowym znajdą się te same dane. Należy tego unikać.

Wstępne przetwarzanie danych przed skierowaniem ich do sieci neuronowej:

1. **Wektoryzacja**. Wszystkie dane wejściowe i etykiety muszą mieć postać tensorów danych zmiennoprzecinkowych (w niektórych przypadkach liczb całkowitych).
2. **Normalizacja**. Kierowanie do sieci danych, których cechy mają różną skalę (np. jedna to wartości 0–1 a druga 500–700) może uniemożliwić znalezienie właściwego rozwiązania przez sieć neuronową. W celu ułatwienia procesu uczenia sieci dane powinny mieć następujące własności:
   - przyjmować małe wartości (zwykle z zakresu 0–1),
   - być homogeniczne – wszystkie cechy powinny przyjmować wartości z tego samego zakresu.
3. **Problem brakujących wartości**. W sieciach neuronowych brakujące wartości można zastępować zerami (w przypadku, gdy 0 nie jest wartością mającą jakieś znaczenie) – sieć nauczy się, że 0 oznacza brak danych i należy je ignorować.
   Jeśli w zbiorze treningowym nie było brakujących wartości, a pojawią się w zbiorze testowym, sieć nie będzie nauczona aby je ignorować. W takiej sytuacji należy wygenerować sztuczne próbki treningowe z brakującymi elementami poprzez kopiowanie niektórych próbek i usuwanie z nich cech, dla których w zbiorze testowym brakuje wartości.

Kodowanie „gorącą jedynką” (ang. **one-hot encoding).**
Technika wykorzystywana m.in. w przetwarzaniu języka naturalnego (NLP – *Natural Language Processing*).
Polega na generowaniu reprezentacji wektorowej wyrazu, zdania lub tekstu.
Aby przekazać do modelu uczenia maszynowego tekst, należy go najpierw zakodować w postaci liczb lub wektorów.

Przykład:
W zdaniu **Karetę ciągną cztery konie** każdy wyraz jest reprezentowany poprzez liczbę 1 na odpowiednim miejscu a 0 na pozostałych: Np.:
karetę wektor [1, 0, 0, 0] **ciągną** → [0, 1, 0, 0] **cztery** → [0, 0, 1, 0] **konie** → [0, 0, 0, 1]
Wtedy zdanie **Karetę ciągną konie** będzie reprezentował wektor: [1, 1, 0, 1]
a **Cztery konie** wektor [0, 0, 1, 1]

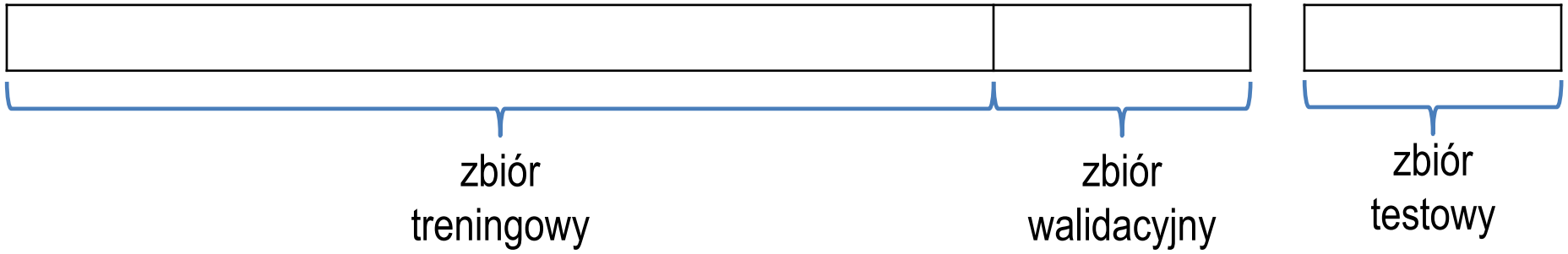
Załóżmy, że mamy 1000 wyrazów w słowniku. Każdy wyraz będzie wtedy reprezentowany przez 1000-elementowy wektor, w którym na odpowiedniej (jednej) pozycji będzie 1, pozostałe będą zerami.
Tego rodzaju reprezentacja nie potrafi uchwycić podobieństwa między wyrazami (iloczyn skalarny dwóch dowolnych wyrazów zawsze wynosi 0).
W zadaniach związanych z przetwarzaniem tekstu możliwość znalezienia podobieństwa wyrazów jest istotna. W uczeniu maszynowym istnieją metody, które to umożliwiają.

Kodowanie „gorącą jedynką” jest także wykorzytywane do eliminacji kolejności w danych, w których takiej kolejnosci nie ma.

Przykład:
zmienna przechowuje kolor oczu w postaci tekstu (zielone, niebieskie, piwne, szare, czarne).
Gdybyśmy każdemu kolorowi przypisali kolejną liczbę (np. zielone 1, niebieskie 2, piwne 3, itd.) model mógłby wysiągnąć wnioski, że między kolorami występuje porządek i jeden jest większy (ważniejszy) od drugiego.
Zamiast tego kodujemy kolory w postaci gorącej jedynki, zastępując nazwę każdego koloru wektorem, w którym na jednym miejscu jest 1, a na pozostałych 0:

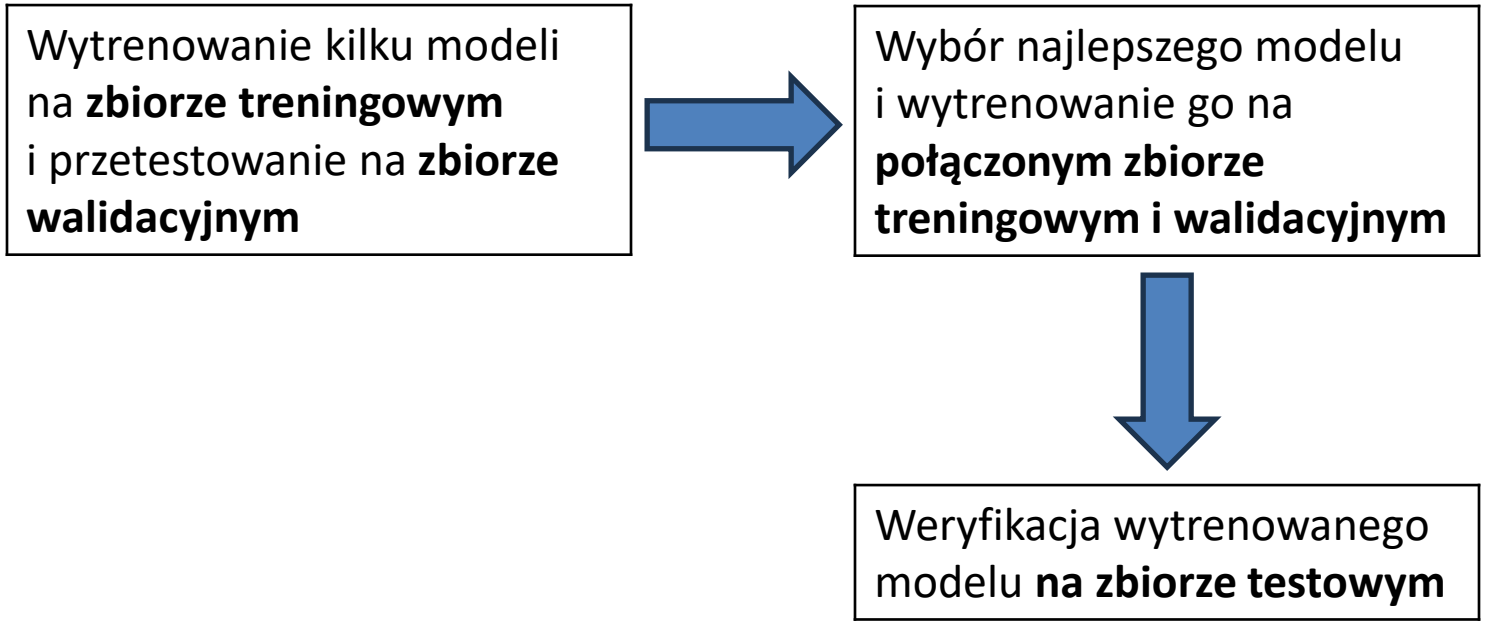
np. zielone → [1, 0, 0, 0, 0]
niebieskie → [0, 1, 0, 0, 0]
…
czarne → [0, 0, 0, 0, 1]

Podział zbioru danych w procesie uczenia maszynowego:



- zbiór treningowy – używany w procesie trenowania modelu,
- zbiór walidacyjny (ewaluacyjny, rozwojowy) – używany do oceny pracy modelu, dostrajania po treningu, dostarcza sygnału zwrotnego na podstawie badania wydajności modelu z wykorzystaniem danych walidacyjnych,
  jest wykorzytywany do wskazania momentu zakończenia uczenia,
- zbiór testowy – służy do weryfikacji poprawności modelu, nie bierze udziału w procesie trenowania i walidacji; musi dobrze odwzorowywać rozkład główny, z którego pochodzą dane (częstość pojawiania się poszczególnych klas w zbiorze testowym powinna być zgodna z oczekiwaną częsością ich występowania w rzeczywistości),

Zbiór **walidacyjny** często jest wykorzystywany w charakterze zbioru testowego podczas trenowania różnych modeli. Najlepszy z wyrenowanych w ten sposób modeli trenuje się z wykorzystaniem połączonego zbioru treningowego i walidacyjnego oraz testuje przy pomocy odłożonego zbioru testowego. W ten sposób zbiór testowy jest wykorzystany tylko raz – do ewaluacji ostatecznie wytrenowanego modelu.



Celem uczenia maszynowego jest uzyskanie modelu zdolnego do **uogólnień** – działającego dobrze na danych, których wcześniej nie „widział”.

W procesie uczenia maszynowego wykorzystywane są algorytmy i modele do przewidywania wyników. Aby osiągnięcie tego celu było możliwe, należy:

- korzystać z najdokładniejszych i przygotowanych wcześniej **danych**,
- w niektórych przypadkach dostarczyć oczekiwanych **wyników**,
- posługiwać się odpowiednimi **algorytmami**,
- tworzyć adekwatne **modele**,
- **mierzyć**, czy modele działają poprawnie.

GPU kontra CPU

CPU (Central Processing Unit)	GPU (Graphical Processing Unit)
kilka rdzeni	setki rdzeni
mogą przetwarzać kilka wątków jednocześnie	mogą przetwarzać setki wątków jednocześnie
jednostki o wysokim stopniu współbieżności	jednostki na ogół szeregowe
optymalizowane pod kątem opóźnień	optymalizowane pod kątem przepustowości, a ponieważ w sieciach neuronowych wąskim gardłem jest przepustowość, więc mają lepszą wydajność
	pamięć podręczna jest znacznie szybsza niż w układzie CPU a ponieważ w sieciach neuronowych duża część danych jest wykorzystywana wielokrotnie, więc pamięć L1* ma znaczenie
	dzięki wykorzystaniu GPU program może działać wielokrotnie szybciej niż w przypadku używania wyłącznie CPU
	CUDA dla NVIDIA – projekt udostępnienia interfejsu programistycznego do procesorów graficznych

*Pamięć podręczna L1 zawiera informacje, które prawdopodobnie będą potrzebne w najbliższej przyszłości.

W Windows, aby sprawdzić liczbę rdzeni procesora należy przycisnąć Ctrl+Sh+Esc i wybrać zakładkę *Wydajność*.

Najpopularniejsze rozwiązania dla ML i DS:

- Apache FlinkML – szybko rosnąca popularność
- Apache OpenNLP
- Apache Spark->MLlib
- Caffe2
- H2O – platforma do DS i ML
- **Scikit-Learn** (2007) – zawiera gotowe klasyfikatory oparte na różnych algorytmach
- **TensorFlow** (Google, 2015)
- **Keras** (2015) – interface początkowo do TensorFlow, obecnie też CNTK, Theano, PlaidML
- **PyTorch** (Facebook, 2016)

Inne, m.in. :

- Caffe (2013)
- **CNTK (2016) – Microsoft Cognitive Toolkit**
- DistBelief (2012)
- Lasagne (na bazie Theano)
- MXNet (2015)
- PyLearn2 (2013)
- **R Project – popularna, choć traci użytkowników na rzecz rozwiązań w Pythonie**
- **SciPy – w Pythonie, wykorzystuje struktury NumPy**
- Theano (2010) – nie rozwijana

Do zarządzania danymi i obliczeń naukowych:

NumPy (numerical python)
Do obliczeń numerycznych dla danych, które mogą być przechowywane w tensorach.
Jest używana w uczeniu maszynowym, wizualizacji danych, analizie danych itp.

Pandas
Służy do analizy danych i zarządzania danymi. Jest używana zwykle z NumPy i Matplotlib.
Obsługuje pliki CSV, korzysta z ramki danych (DataFrame) i jest szczególnie przydatna w procesie przygotowania danych.

SciPy (scientific python)
Wykorzystywana w projektach obejmujących różne obliczenia: matematyczne, statystyczne, inżynieryjne. Współpracuje z Matplotlib, NumPy, SymPy, Pandas, iPython itp.

Do wizualizacji danych

Matplotlib

Do wizualizacji danych w języku Python. Umożliwia tworzenie histogramów, wykresów rozrzutu, słupkowych, kołowych, itp. Wraz z NumPy jest używana do rozwiązywania zadań matematycznych.

Seaborn

Do wizualizacji danych opartą na Matplotlib i dostarcza metod do wizualizacji statystycznych. Jest zintegrowana z NumPy i Pandas. Ma funkcje do operowania na ramkach danych (DataFrame) i tablicach. Zawiera ogromny zbiór narzędzi do złożonej wizualizacji. Umożliwia rysowanie wykresów kołowych, histogramów, wykresów rozrzutu, słupkowych itp.

Do uczenia maszynowego i uczenia głębokiego

Scikit-learn (sklearn, Google)
Używana do rozwiązywania różnych problemów, które obejmują regresję, klasyfikację. Dostarcza gotowych klasyfikatorów dla takich metod jak: jak maszyny wektorów nośnych, lasy losowe, kNN, naiwne klasyfikatory Bayesa, drzewa decyzyjne i wiele innych.

TensorFlow (Google)
Tensorflow to biblioteka pythonowa do implementacji sieci głębokich. Służy do opracowywania i wdrażania modeli uczenia maszynowego za pomocą wysokopoziomowych interfejsów API, takich jak Keras. Za pomocą TensorFlow można zaimplementować CNN (Convolutional neural network) i RNN (Recurrent neural networks).

Keras (autor: François Chollet, inżynier Google)
Biblioteka do budowania sieci neuronowych. Dostarcza zoptymalizowane rozwiązania. Pozwala na szybszą iterację i wdrażanie modeli niż biblioteki, do których stanowi interface. Działa jako interfejs dla Tensorflow (od wersji 2.4 także innych).

PyTorch (Facebook)
Biblioteka Pythona, która służy do tworzenia modeli uczenia głębokiego. Została opracowana w laboratorium badawczym Facebooka. Jest wykorzystywana do wykonywania operacji na tensorach. Obsługuje obliczenia tensorowe z akceleracją GPU (*graphics processing unit*).

Keras – własności:

- możliwość uruchamiania kodu na procesorach CPU i GPU,
- przyjazny interfejs programistyczny,
- wbudowana obsługa sieci konwolucyjnych (do przetwarzania obrazu), rekurencyjnych (do przetwarzania danych sekwencyjnych) oraz sieci będących połączeniem obu tych rozwiązań,
- obsługa sieci o dowolnych architekturach,
- kod może być uruchomiony bez modyfikacji na różnych platformach (Theano, CNTK, **TensorFlow**),
- jej wykorzystanie pozwala na uproszczenie kodu w porównaniu z wykorzystaniem platform, na bazie których działa (Theano, CNTK, TensorFlow).



Środowiska do ML w Python:

Anaconda

- najpopularniejsza platforma desktopowa do obliczeń naukowych w R i Python
- umożliwia łatwe zarządzanie pakietami oprogramowania (kilkaset pakietów instalowanych automatycznie)
- dostępna na Windows, Linux i macOS

Goodle COLABORATORY https://colab.research.google.com

- środowisko „przeglądarkowe” (chmurowy Jupyter)
- obliczenia wykonywane w chmurze, pliki zapisywane na dysku google
- umożliwia wykonywanie obliczeń z wykorzystaniem GPU i TPU (Tensor Processing Units, wspomaganie dla TensorFlow, może znacznie skrócić czas uczenia modeli, nawet w porównaniu z GPU)
  GPU/TPU szczegóły: https://cloud.google.com/tpu/docs/tpus
  Warunki i ograniczenia na stronie:
  https://research.google.com/colaboratory/faq.html#resource-limits

Inne:
PyCharm, Visual Studio, Sublime Text, VIM, IDLE, …
oraz, dostępne m.in. w środowisku Anacondy:
Jupyter, Spider